

修士論文概要書

Master's Thesis Summary

Date of submission: 07/20/2026 (MM/DD/YYYY)

専攻名 (専攻分野) Department	情報理工・ 情報通信専攻	氏名 Name	Takumi Otsuka	指 導 員 教 員 Advisor	Kazue Sako 印 Seal
研究指導名 Research guidance	暗号プロトコル 研究	学籍番号 Student ID number	5124FG15		
研究題目 Title	Locating the Deployability Wall for Post-Quantum Anonymous Credentials in General-Purpose zkVMs				

1 Introduction

Post-quantum anonymous credentials let a holder prove possession of a valid credential without revealing identity, under assumptions that survive a quantum adversary. We target the setting in which the holder generates this proof on a *personal device*, here a consumer laptop (18-core, 24 GB), as opposed to a cloud or server prover. A general-purpose zero-knowledge virtual machine (zkVM) is attractive in this setting for *flexibility* rather than for raw speed: it proves an arbitrary verification predicate without a bespoke, per-scheme circuit. The obstacle is arithmetic. The schemes we care about (PLUM, Loquat) operate over a 192-bit prime field, while production zkVMs (RISC Zero, SP1) fix a small (≈ 31 -bit) native field, so every operation is emulated by multi-limb arithmetic; an algebraic hash (Griffin), which makes up $\approx 91\%$ of the verification's R1CS constraints at PLUM-128, amplifies the penalty. This thesis asks whether such a verification can be made to run on a personal device at all, what it costs, and whether the resulting proof can additionally be zero-knowledge.

2 Theoretical Foundations of The Arithmetic Mismatch

We formalize *arithmetic mismatch* as the performance gap that arises when a cryptographic primitive's natural algebraic domain (e.g., arithmetic over a large prime field $\mathbb{F}_q$ or structured rings) does not coincide with the proof system's *native arithmetic*. We model a proof backend by its *arithmetization regime*: (i) a native field $\mathbb{F}_p$, (ii) an encoding of the computation domain into $\mathbb{F}_p$ (possibly via multi-limb representations), and (iii) a constraint/execution representation (e.g., R1CS for static circuits or AIR/trace constraints for STARK-based zkVMs). This viewpoint separates two qualitatively different regimes.

In the *field-aligned* regime typical of application-specific static circuits, the designer can select (or align to) a proof system whose scalar field matches the primitive's domain, so arithmetic operations are expressed directly: multiplications correspond to constant-cost constraints, and total constraint growth tracks the primitive's multiplicative complexity. In the *mismatch* regime typical of general-purpose zkVMs, the native field $\mathbb{F}_{\text{pvm}}$ is fixed by the VM/STARK design and is often much smaller than $\mathbb{F}_q$. Elements of $\mathbb{F}_q$ must then be represented as $k = \lceil \log_2(q)/w \rceil$ w -bit limbs, requiring carry propagation and

modular reduction to emulate non-native arithmetic. We show that even a single non-native modular multiplication is lower-bounded by $\Omega(k)$ native multiplications (with schoolbook emulation realizing all k^2 cross-products, plus carries and reduction), yielding an inherent "agility tax" that cannot be eliminated by constant-factor engineering alone.

For STARK-based zkVMs, this cost further amplifies at the proof-system level: limb-based arithmetic increases the executed instruction count and inflates the AIR execution trace length by $\Theta(k^2)$ under the schoolbook emulation the evaluated zkVMs use. Since zkVM prover time is dominated by low-degree testing and commitments and scales roughly as $O(W \cdot T \log T)$ in trace width W and length T , arithmetic mismatch translates into a fundamental prover slowdown relative to field-aligned static circuits. We also note a practical implication for system design: zkVM ecosystems may accelerate standardized primitives via precompiles or specialized gadgets, so custom arithmetisation-oriented primitives can become comparatively slower in zkVMs when they lack such accelerations (the *precompile paradox*).

3 System Design

We construct a custom *precompile suite* that bypasses field emulation for the primitives that dominate the cost, mirroring how zkVMs already ship precompiles for standardized primitives (e.g. SHA-2, secp256k1). Three AIR modules are specified, in expected cycle-share order: (i) the Griffin permutation over PLUM's 192-bit prime field, the load-bearing 91% target; (ii) multi-limb 192-bit modular arithmetic over $p = 2^{64}p_0 + 1$; and (iii) the t -th power-residue PRF symbol check ($t = 256$ fixed), a constant-exponent routine for $a^{(p-1)/t} \bmod p$. Each module ships with a written soundness argument tying its AIR constraints to the reference specification, so the precompile layer preserves the scheme's unforgeability under the assumption that the zkVM's cross-table lookup argument is sound.

4 Evaluation

We measure four cells on the target hardware: PLUM verification in a zkVM (1) without and (2) with the Griffin precompile; (3) the same verification using software SHA-3 as a control; and (4) Loquat verification in an application-specific circuit (Aurora) as an in-SNARK reference (the measured static baseline; PLUM over its 199-bit field has no Aurora harness).

Without the precompile, verification does not terminate within a 24 GB/3 h bound (≈ 7.08 billion cycles at $\lambda = 80$, projecting to tens of hours). With the precompile it completes in 14.25 min (mean of three fixed-config runs, 14.05–14.61) on the 24 GB machine at a peak ≈ 14.2 GB resident, the first configuration in the study to fit the budget.

The control is the sharpest result. Even with the precompile, PLUM verification executes $1.13\times$ more cycles (125.5 M vs 110.8 M) and proves about $1.09\times$ slower in wall-clock (14.25 vs 13.05 min, mean of three): a standardized hash, unaccelerated, still beats our accelerated algebraic hash. This is the *precompile paradox* made empirical: the cost of choosing a bespoke algebraic-hash primitive over one the ecosystem has standardized.

5 Discussion

Feasibility is not deployability. A succinct proof on this substrate is not zero-knowledge: SP1’s default STARK proof is succinct but not ZK, and ZK is obtained only by wrapping it in a Groth16/PLONK SNARK over a pairing-friendly curve, which is *not* post-quantum, so a future quantum adversary retroactively de-anonymizes every stored transcript. The post-quantum ZK route (a masked zk-STARK) yields only *computational* post-quantum anonymity in the (Q)ROM, not everlasting (information-theoretic) anonymity, because the witness trace is committed under a computationally-hiding hash. No available zkVM path delivers post-quantum soundness and zero-knowledge jointly. This *deployability wall*, rather than the prover’s speed, is the binding obstacle for a personal-device, post-quantum anonymous credential.

6 Conclusion

We formalized the arithmetic mismatch as a quantifiable agility tax, specified a precompile suite that brings post-quantum anonymous-credential verification within the personal-device memory budget on this substrate, and showed both an empirical instance of the precompile paradox and a deployability wall that blocks a jointly post-quantum and zero-knowledge proof. Closing that wall (an everlasting-anonymous proof on a transparent post-quantum substrate, and pushing below the personal-device budget toward mobile) is left as future work.